Design (E) 314
Project Definition

Ontwerp (E) 314
Projek Definisie

2026

Dr Armand du Plessis, Prof Herman Engelbrecht

This document is NOT the final version. Further information will be added throughout the course.

Version History

Version	Date	Changes
0.1	9 Feb 2026	- Initial project specification and guide

1 Purpose of this Document

The purpose of this document is to:

- Provide students with a clear **overview and scope** definition of the project;
- Provide clear **project requirements**, that will be used to test the hardware during demonstrations;
- Provide some assistance in understanding certain **concepts/information** about the components that will used in the project;
- Identify the **critical design choices** that the student should solve.
- Provide **guidance for producing the final report** using an assessment rubric.

For other information about the course ***schedule*** and ***administration***, please see the Module Framework available on STEMLearn.

2 Project Overview

2.1 Embedded Driver Assistance System (E-DAS)

Road safety remains a critical concern, with many vehicles lacking the intelligent assistance features of modern cars. The E-DAS project aims to demonstrate how a compact, affordable embedded system can enhance driver safety and awareness through real-time monitoring and data-driven insights. The motivation is twofold:

- Safety Enhancement: Providing drivers with proximity, and environmental feedback to prevent accidents.
- Data Analytics: Logging and analysing vehicle and driver performance to promote safer, more efficient driving habits.

This project showcases how sensor fusion, embedded control, and analytics can come together in an impactful, real-world engineering solution.

The E-DAS is a self-powered embedded system designed to plug into a vehicle's 12 V outlet socket. It continuously monitors distance to nearby objects, vehicle motion, environmental light, and temperature, displaying real-time, visual feedback to the driver. The device also logs sensor and location data to a storage device for later analysis, while an human-computer interface allows the driver to input parameters (e.g., fuel tank capacity) and navigate menus.

2.2 From user requirements to system specifications

The project will consist of designing, building, and testing a multi-functional device, with the primary objective of reporting on monitoring and logging driver behaviour and vehicle conditions.

The high-level user requirements and core functions of the E-DAS can be derived from the description in section 2.1 as follows:

A. Safety and Awareness
- Proximity Detection: the system measures and records the distance to obstacles, and displays visual proximity warnings.
- Accident/Unsafe Driving Detection: the system measures and records the acceleration of the vehicle and to detect rapid acceleration, deceleration and/or impacts.
- Light Monitoring: the system measures and records environmental light conditions, and displays a visual warning when headlights are needed;
- Temperature Monitoring: the system measures and records the environmental temperature, and displays a visual warning when uncomfortable temperature is reached.

B. Analytics and Logging
- Location Logging: the system measures and records distance travelled, and location data for performance analysis.

- Fuel Efficiency Estimation: the system uses user-supplied fuel capacity and location data to estimate fuel consumption efficiency.
- All sensor measurement data are continuously recorded onto a storage device at specific intervals.
- Unpredictable events (e.g. accidents, proximity warnings) are recorded when the event occurs.

C. User Interface

- Visual Display: Real-time readings, warnings, and user menu navigation.
- Keypad and Button Input: User Menu control and parameter entry.
- LED Indicators: Visual alerts for proximity, lighting, temperature, driving and impact detected, and system status.

D. Diagnostics

- Device communication: Data extraction and debugging via UART (using the STM32 microprocessor's USB-UART interface).

From the above high-level user requirements, the system specifications are detailed in Table 1.

Table 1: System Specifications (SS) for the project.

SS#	Description
SS1	The system must generate its own regulated supplies from a nominal 9 V-12 V battery or power supply. 1. 5 V supply voltage 2. 3.3 V supply voltage 3. LED (D6) must turn on to indicate that the 5V regulated voltage is present.
SS2	The device must use a UART connection, in both transmit and receive modes. The device shall operate in 8 data bits, one even parity bit, and one stop bits configuration, at a data rate of 57600 baud. The UART information, both transmitted and received, shall be formatted as per section 4.6.1 and 4.6.2. The system shall receive commands as defined in section 4.6.1 and respond accordingly. 1. No sooner than 100 ms after startup, and no later than 500 ms after startup, the device will send the board owner's student number, formatted as per section 4.6.1. 2. The device shall be able to handle commands and respond with information and data as detailed in Section 4.6.1 and 4.6.2.
SS3	An ultrasonic sensor is used to measure distance. The device must determine the distance from sensor to an object as detailed in Section 3.6 to at least 30cm. If an object is within 10cm of the ultrasonic sensor a proximity warning must be displayed by flashing the appropriate indicator LED.
SS4	An Micro-Electro-Mechanical Systems (MEMS) sensor is used to measure acceleration. The device must record the measured acceleration, and perform unsafe driving and impact detection as detailed in Section 3.11. Unsafe driving is defined when the measured acceleration (excluding gravity) exceeds 0.5g in any direction. An impact is detected when the measured acceleration (excluding gravity) exceeds 1.5g in any direction. If unsafe driving is detected then an unsafe driving warning must be displayed by flashing the appropriate indicator LED, and displaying a warning message on the LCD. If an impact is detected then a impact warning must be displayed by switching on the appropriate indicator LED, and displaying a warning message on the LCD.
SS5	A photodiode is used to measure the ambient light conditions. The device must record the light conditions, and perform low-light detection as detailed in Section 3.8. A low-light condition is defined when the measured light intensity is less than 300 lux. If a low-light condition is detected, a low-light warning must be displayed by flashing the appropriate indicator LED, and displaying a warning message on the LCD.
	Continued on next page

	Table 1 – continued from previous page
UR#	**Description**
SS6	A temperature sensor is used to measure the ambient temperature. The device must record the temperature, and determine if the temperature reaches uncomfortable levels, as detailed in Section 3.7. Uncomfortable temperature is defined as measured temperature of more than 30°C. If an uncomfortable temperature is reached, a temperature warning must be displayed by flashing the appropriate indicator LED, and displaying a warning message on the LCD.
SS7	The device must use a set of status-indicating LEDs as output devices as detailed in Section 4.5. D2 (proximity warning), D3 (unsafe driving and impact detected), D4 (low-light conditions) and D5 (uncomfortable temperature) will act as Alarm indicators. The flashing timing for the status-indicating LEDs must be 500 ms ON and 500 ms OFF.
SS8	The device must use a keypad as user input for various system settings and tactile buttons to navigate display modes as detailed in Sections 3.4, 3.9 and 4.13.
SS9	The device must use an LCD display as the primary visual output device to inform the user, display warning message, and allow the user to navigate the user interface, as detailed in Sections 3.10 and 4.14.
SS10	The device must record the measured and calculated data to an SD card as detailed in section 4.15.
SS11	The device must keep accurate time using an RTC (internal to the microprocessor) as detailed in Section 4.8.
SS12	The device must implement a Test Interface Connection (TIC) and debug interfaces, including a push button input, as detailed in section 5.1.

For more detail about how these specifications will be tested, see section 5

2.3 Building the prototype

You will be provided with a ***baseboard*** and an STM32 microcontroller board. The microcontroller board will connect to headers, which have to be soldered onto the baseboard. Some connections to the power supply are already integrated into the baseboard. You will have to design certain elements of the system and make decisions about wiring and electrical connections.

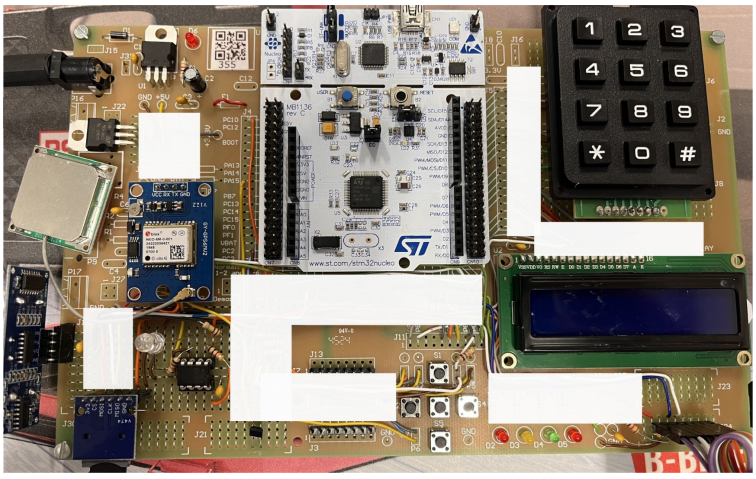


Figure 1: An example of the E-DAS prototype board.

You will also have to devise suitable tests to show that your system conforms to the mentioned requirements and specifications. Your design process, decisions, and justification, along with test methods and test results should be documented in a final report, which is due at the end of this module.

Summary of Specifications for Demo 1

Your system will be tested using automated demonstration stations, which will test whether your system meets these system specifications. It is thus important that you devise and perform similar tests to what the demo station will run, ***before*** you present your board for a demonstration.

> ⚠ *The demonstration should not be used as a substitute for your own testing since you will be penalised for multiple demonstrations.*

The rest of this document describes the specifications in more detail, the hardware that you will be required to use and interface to (section 3), as well as software considerations (section 4), and finally the test methods that will be used to test your board (section 5).

2.4 UART communication

The system should use the UART port connected to the PC (**and the TIC for Demo purposes** see section 5.3) to report measurements and system statuses and to receive commands. The full details of the communication protocol are given in section 4.6.

2.5 Application command interface

The system should implement an application command interface allowing the user to select the operating mode and modify system parameters. The command input interface shall be via the push button, keypad, and UART. The output interface shall be via the UART, LCD, SD card, and status LEDs.

3 Hardware

The STM32 microcontroller board that you will use is the STM32F411RE. It has a NUCLEO-64 form factor which allows it to stack through either the Arduino headers or the Morpho connectors (on top). The STM board will stack on top of the prototyping baseboard that you will use. To simplify the connection choices and possible solder problems, the following pre-determined design choices were made:

- The digital and analogue ground pins of the microcontroller board are hard-wired to the main board ground
- The NUCLEO-F411RE is provided with main board power via the 'E5V' pin (CN7-6) and F1 (fuse or link).
- To assist in building the system, some peripherals have pre-defined layout positions. These include:
  - 5 V power regulation circuit (3.3 V power regulation circuit has a predefined prototyping area where it should be built)
  - Debug / user indicator LEDs
  - Debug / user push button switches
  - Test interface connector (used for Demonstrations)
  - Miscellaneous connectors that might be applicable to this project, eg. power socket, USB, screw terminal, audio jack.

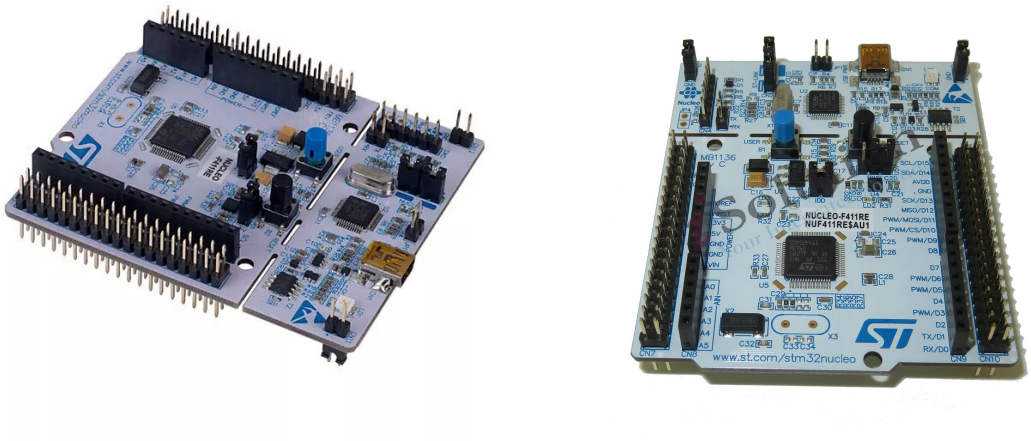


Figure 2: NUCLEO-F411RE development board [2]

3.1 Power Supply

The board must be supplied with a nominal 9 V DC source (but you can use a bench power supply during development and testing). Your project will require you to implement both a 5 V and a 3.3 V regulated supply. The 5 V circuit is pre-designed and shown in figure 3.

The 5 V power supply circuit on the baseboard uses a standard 7805 regulator (U1) in a TO220 package to regulate the input down to 5 V. The 5 V power is routed to 3 jumpers (J14, J16 and J25, each providing 3 pins for wiring). An LED (D6) circuit is provided to show that a voltage is present on the 5 V line.

> *You will have to determine the minimum supply voltage input required as well as the maximum allowable input voltage. You must also make sure that the regulator (U1) is thermally stable (by calculating the expected heat dissipation and adding thermal heat sinking hardware, if required). You must understand the role of each component in the power supply circuit.*

You will have to design the 3.3 V supply circuit yourself using the L78L33 regulator, provided in a TO-92

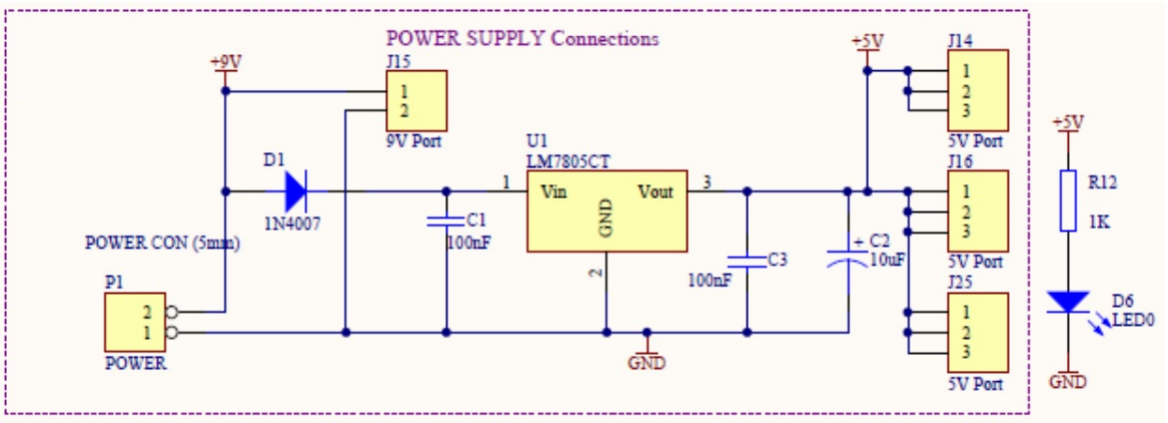
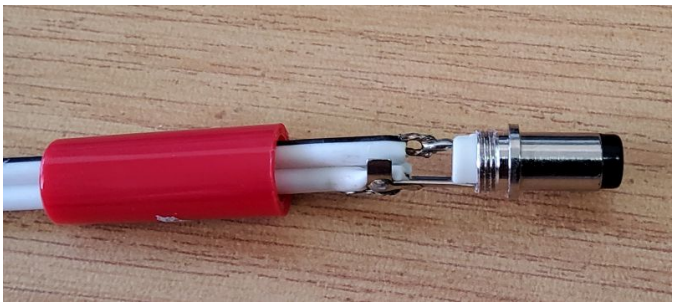


Figure 3: 5 V Power supply circuit diagram

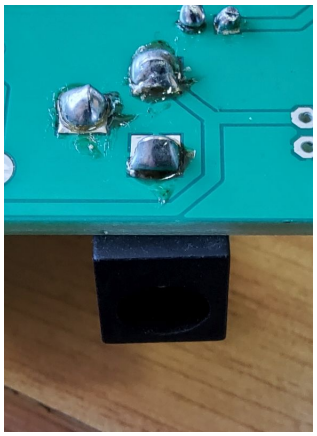
package. Available on the baseboard, there is a 3.3 V power rail routed to 3 jumpers (J17, J18 and J26, each providing 3 pins for wiring).

> ⚠ *Note that the 3.3 V supply should NOT be connected to the NUCLEO-F411RE's own 3.3 V regulated voltage!*

On your NUCLEO board, the `VIN` pin should be left floating. To select board power, move the JP2 jumper on the NUCLEO-F411RE to the `E5V` (external power) position, from the `U5V` (USB power) position. Power your board using a bench power supply, set to 9 V, and connect the power to your board using the barrel jack and wire provided.



(a) Barrel jack with wires soldered



(b) Barrel socket soldered on to the PCB

Figure 4: Power connector solder connections.

> ⚠ *After your power supply is built DO NOT power your baseboard and NUCLEO system using the USB power from the PC, as this could damage the USB port. It is safer to keep the jumper in the* `E5V` *position and use an external 9 V power supply. See more detail on this in "UM1724 User manual" [3] PDF document, section 7.5 on page 21 - 23.*

> ⓘ *Jumper J30 must be bridged on baseboard to allow the test station to supply the student board with a nominal 9 V during demonstrations. Jumper J31 must be bridged to supply 9 V to the regulator.*

The fuse, F1, must be bridged on student boards to supply the STM32 board with 5 V.

3.2 NUCLEO-F411RE Connections

The NUCLEO-F411RE plugs into the baseboard through two separate 38-pin double-row connectors, CN7 (left) and CN10 (right). These connections correspond to the NUCLEO Morpho connectors. The outer row connections are linked to baseboard solder connections J4 (for the odd-numbered CN7 pins) and J5 (for the even-numbered CN10 pins). The majority of the inner row CN7 (even-numbered) pins that correspond to Arduino connections are brought out through J10, and the odd-numbered CN10 pins are brought out through J9 and J11.

The 2021 version of the baseboard does not indicate the NUCLEO pinouts correctly. The NUCLEO-F411RE's board layout - MB1136 - differs somewhat from the previous year's NUCLEO board layout. Therefore some of the pin names indicated on the baseboard, see figure 5, are correct but others might not be. You must check the pin names against the pin layout of the NUCLEO-F411RE as indicated in [3], pages 33, 44-45.

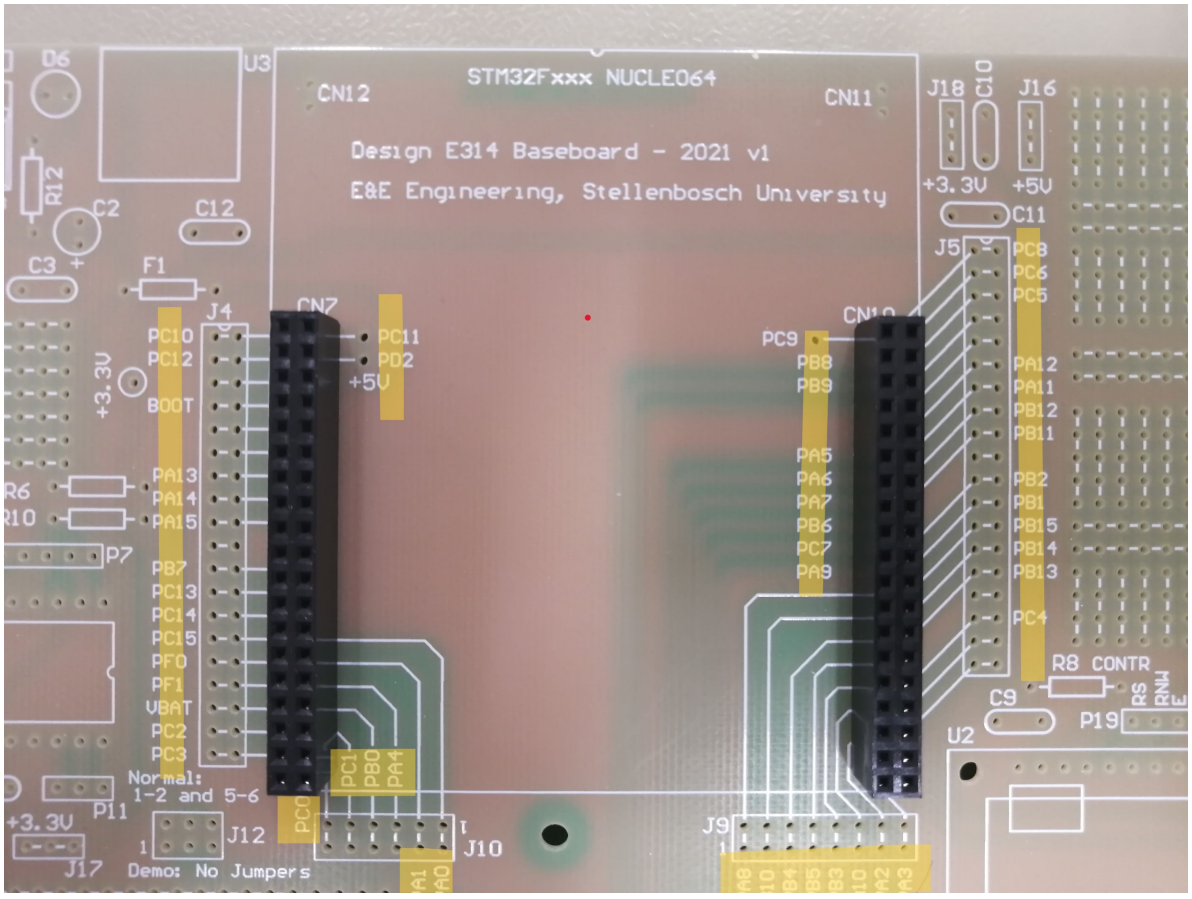


Figure 5: The baseboard indicating the pin names that must be checked.

It is good practice to trace the PCB connections on your board and familiarise yourself with the connections made via the PCB and therefore the interfaces available to you.

You will need to refer to the pin information in user manual UM1724 (pin assignment and connectors) whenever you need to decide on which NUCLEO-F411RE pin to use for a specific function. Some peripherals have only one or two options, so where you have the design freedom, make sure you plan for future possible pin uses too.

> ⚠ *DO NOT CONNECT the following pins of the NUCLEO-F411RE, in your application unless you have a specific requirement - `BOOT`, `5V`, `RESET`, `VIN` and any of the `NC` pins. The NUCLEO-F411RE operates normally without connections to these pins.*

3.3 Debug / user indicator LEDs

Space for four LEDs plus series resistors are provided for display and/or debug purposes. You will have to calculate an appropriate value for the series resistors to limit the LED current to an acceptable level (refer to the datasheet). These LEDs must be driven by four GPIO pins of the Nucleo board. The details of the state/functionality that is represented by each LED (or LED combination) is given in section 4.5.

> ⓘ *These LED circuit pins are also floating on the baseboard - they are not connected to any other signal. You need to wire them to the pins/signals you wish to use.*

> ⚠ *Remember to limit the current through the LEDs by using a series resistor, otherwise you may damage the processor output pins!*

3.4 Push button input

Space for five tactile push-buttons (as shown in Fig. 6) are provided on the board. They are logically arranged to represent up, down, left, right and middle. You need to wire the buttons to be active low, and read as inputs by the GPIO pins of the Nucleo board. They will also need to be connected to the TIC as per Table 8.

Figure 6: Tactile push-button.

The buttons can be connected in either an active high or active low configuration, as shown in Fig. 7. For your project, you must connect the buttons in active high configuration so that when a key is pressed, a high input is read. When no key is pressed, the input should read low.

Debouncing each button input would be good practice to prevent erroneous double-press events.

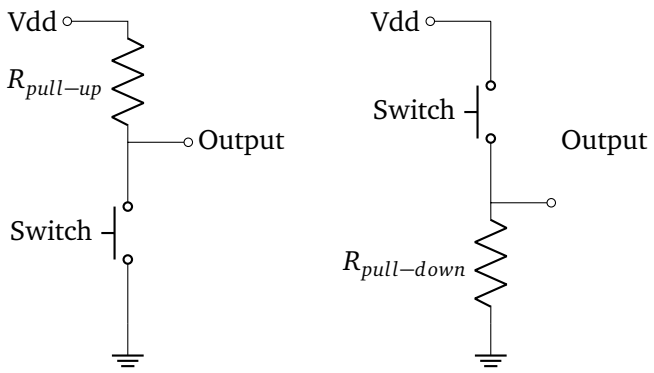


Figure 7: Circuits showing active low vs active high configuration.

These pins are also floating on the baseboard - they are not connected to any other signal. You need to wire them to the pins/signals you wish to use.

Limit the current through the buttons by using a series resistor, otherwise you will damage your regulator circuit!

3.5 Test-interface Connector

A test-interface connector (TIC) is provided by P13. This connector is designed to be stackable and consists of an Amphenol Bergstik 16-pin surface mounted connector soldered to the bottom of the baseboard (solder side - see Figure 8). This connector mates with a 16-pin Amphenol Dubox connector on the Automated Test board (top side) that will be used during demonstrations. The connections provide a power source (9 V), UART and 10 other connections during testing of the system.

This connector is supported by two solder connectors (J3 and J13) to connect to various ports or circuits.

More detail about the TIC, and its complete pin descriptions are given in Section 5.

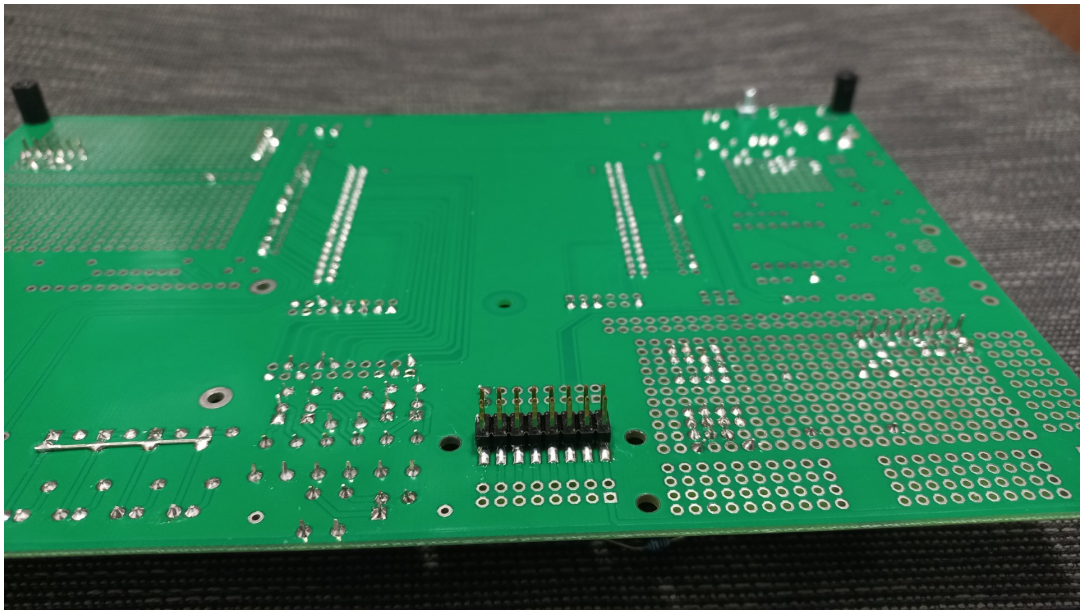


Figure 8: The TIC shown on the solder side of the baseboard.

3.6 Ultrasonic Sensor

The system will measure the distance to objects using the HC-SR04 ultrasonic sensor. The sensor contains both an ultrasonic transmitter and receiver, and uses sonar to measure the distance to an object. The pinout of the HC-SR04 is shown in Fig. 9.

The distance to an object is determined by transmitting a 40kHz ultrasonic pulse when the sensor is triggered. The sensor is triggered by setting the **Trig** pin high (5V) for a minimum of 10 microseconds. The ultrasonic pulse is reflected by the closest object, and received by the receiver. When the sensor detects an ultrasonic pulse is received, it sets the **Echo** pin high to generate a rectangular pulse that has a duration that is proportional to the distance the pulse has travelled. The duration of the echo pulse can be used to determine the total distance of the object from the ultrasonic sensor.

The ultrasonic sensors need to be connected to two GPIO pins of the Nucleo board. One GPIO pin should be used to periodically initiate an ultrasonic pulse, while the other GPIO pin should be used to measure the

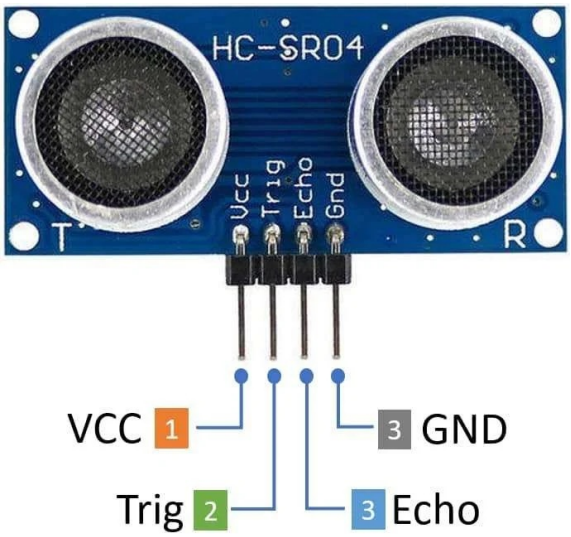


Figure 9: HC-SR04 Ultrasonic Sensor Pinout

duration (width) of the echo pulse. You would need some way to detect both rising and falling edges of the echo pulse and interrupt the STM32 to accurately measure the Echo pulse duration. Refer the datasheet of the HC-SR04 on STEMLearn for more detail.

3.7 Temperature sensing

The system will measure the ambient temperatures using the LMT01 digital temperature sensor (shown in Fig. 10).

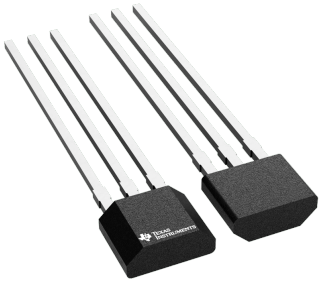


Figure 10: LMT01 digital temperature sensor.

> ⚠ *The temperature sensors are sensitive devices, please handle it with care and do not expose it to harsh conditions or over voltages.*

This sensor has a two wire interface that you connect using GPIO pins and a resistor. Please see the datasheet of the LMT01 device on STEMLearn for more information. This sensor uses a digital pulse train to communicate the measured temperature within a timing window. You will have to count the pulses using a GPIO pin. This can be done by using the GPIO pin as a "clock" input, and the pulse counting can be accomplished by configuring one of the STM32F411 timers. Alternatively, configuring the GPIO pin with an external interrupt and counting in software is also possible. In both cases you will potentially need a second timer to handle the window timing.

3.8 Light sensing

The system will measure the ambient light intensity using a photo sensitive diode-based sensor (as shown in Fig. 11).

Figure 11: Osram SFH 203 PIN photodiod.

Measurements are to be made with an Osram SFH 203 PIN photodiode and an operational amplifier by creating a photo-current-to-voltage converter circuit. The output voltage should be wired to an ADC pin of the STM32. The output should also be wired to TIC J13 pin 12 as detailed in Table 8. The diagram shown in figure 12 illustrates the recommended design for the photo-current-to-voltage converter circuit. The only design choice you have to make is choosing an appropriate feedback resistor R_f. The MCP602 contains two built-in op-amps. For this circuit you will only need to use one of them.

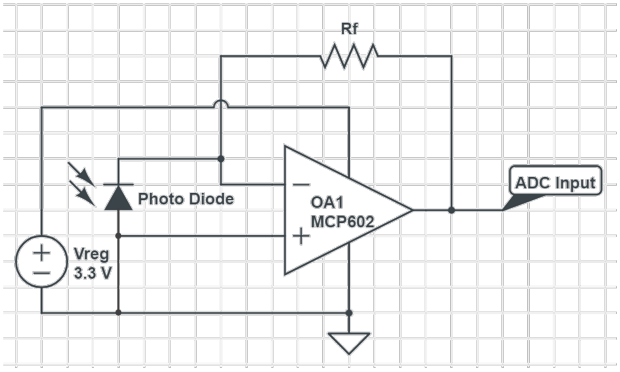


Figure 12: Photo diode with Op-Amp circuit diagram - in Photovoltaic mode.

3.9 Keypad

The Keypad you will use has a 4 (rows) x 3 (columns) matrix layout. You can use a multimeter to verify each row and column connection. The typical layout is indicated in the diagram below. The pinout of the keypad is described in the datasheet on STEMLearn.

Figure 13: 3x4 keypad.

You should connect either the rows or columns to GPIO outputs and the other (rows or columns) as GPIO

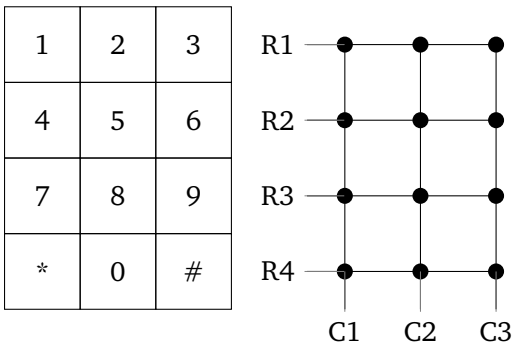


Figure 14: Diagram indicating how the keypad is laid out in 4 rows and 3 columns. Each dot indicates a connection that is closed when the key is pressed.

inputs to your NUCLEO board. In this way you can "read" which key is pressed. See Section 4.13 for more details on the software driver for the keypad.

Since each key is effectively a button, debouncing each input would be good practice to prevent erroneous double-press events. It will simplify your circuit and code if you use the internally available pull-up/pull-down resistor configuration for the input pins and use the pins in external interrupt mode.

The concept of connecting each Row/Column as an input can be done in either an active high or active low configuration, as shown in figure 7 below. For your project, you must connect the rows and columns in active high configuration so that when a key is pressed, a high input is read. When no key is pressed, the input should read low.

3.10 LCD Display

The system uses the Micro Robotics LCD1602-WB-33V (shown in Fig. 15).

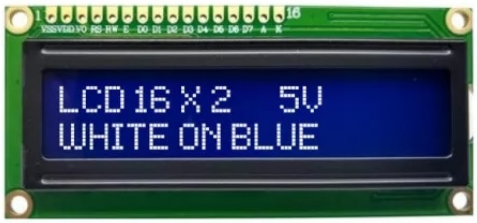


Figure 15: 16x2 character Liquid Crystal Display (LCD).

Each screen has a 16x2 character LCD display (16 characters, two lines), with white text on a blue background. It uses the Hitachi HD44780 controller/driver parallel interface chipset. The following is important to note:

- For these LCDs, the 5V power supply connection can be utilised and no PCB changes are required.
- You have to design and populate the contrast controlling resistors on the mainboard.

You can use a male header strip to solder the LCD to your base board. The LCD connections on the PCB are pre-connected to the required VCC (5 V) and GND. Note that both LCDs are 3.3 V compatible, but the baseboard is powering it from 5 V.

You will need to connect the interface LCD pins electrically to your NUCLEO pins, using wires, from P19.

> *You should only require 7 GPIO pins. You can have all LCD GPIOs in Output mode as you will not be reading data from the LCD.*

You will have to use the LCD in 4-bit (nibble) interface mode, since the LCD's data lines D0 - D3 are hard wired to ground. Please see the LCD driver (Section 4.14) for more detail.

The LCD further has a contrast adjustment pin, for which you should design the two resistors R8 and R9 required to provide a good contrast on the display.

> ⚠ *Do not connect the backlight A and K terminals directly between VCC (5 V) and ground! It is still an LED and it requires a current limiting resistor in series (keep the current below 10 mA). The typical voltage drop across the backlight can be as high as 4.1 V, but be safe and measure it in your circuit.*

If desired, you can connect the LCD backlight for increased visibility. However, there is no specific allowance for connecting the backlight (LED) to the PCB, you will have to connect wires to the A and K solder terminals (Pin 15 and 16). Do not connect the backlight A and K terminals directly between VCC (5 V) and ground! It is still an LED and it requires a current limiting resistor in series (keep the current below 10 mA).

3.11 Accelerometer

The system will use the popular MPU-6050 integrated circuit to measure the vehicle acceleration, which is shown in Fig. 16. The MPU-6050 is a popular, low-cost 6-axis motion tracking device combining a 3-axis accelerometer and a 3-axis gyroscope on one chip. It uses Micro-Electro-Mechanical Systems (MEMS) technology to measure acceleration, tilt, rotation speed, and temperature. The MPU-6050 must be interfaced with the Nucleo board by using the I2C communications protocol.

Figure 16: MPU-6050 motion tracking device

3.12 SD Card module

You will use a generic SD card module (shown in Fig. 17) as an adapter to connect your SD card to the STM32 microprocessor. The SD card uses an SPI interface. You will also have to connect the module's power and chip select (CS) signal to your board.

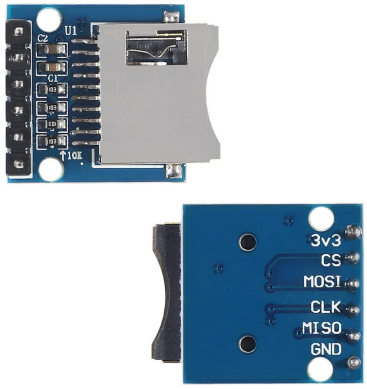


Figure 17: SD card module.

3.13 (Optional) GPS module

The details of the optional GPS module will be detailed in a future version of the Project Definition.

4 Software Design and Considerations

4.1 System operational overview

As the system operates and receives user inputs (via the buttons, keypad, and UART), it should perform the requested commands and output the required information via UART, status LEDs, SD card, and LCD.

4.2 General software information

To develop software for this project, a similar environment and typical development process as used in Computer Systems 245 will be used: STMCubeIDE v1.19.1. There is however one significant change: The use of a software version control system: GIT, described in more detail in section 4.3.

> ⚠ *The use of STM32CubeIDE v1.19.0, with F4 Firmware package v1.28.3 is NOT optional. We cannot offer support for other versions.*
> *When working on the lab PCs, the "firmware updater" path must be set to C:\Progs\STM32CubeIDE_1_19_0\repository. Follow the instructions displayed in Figure 18 and 19.*

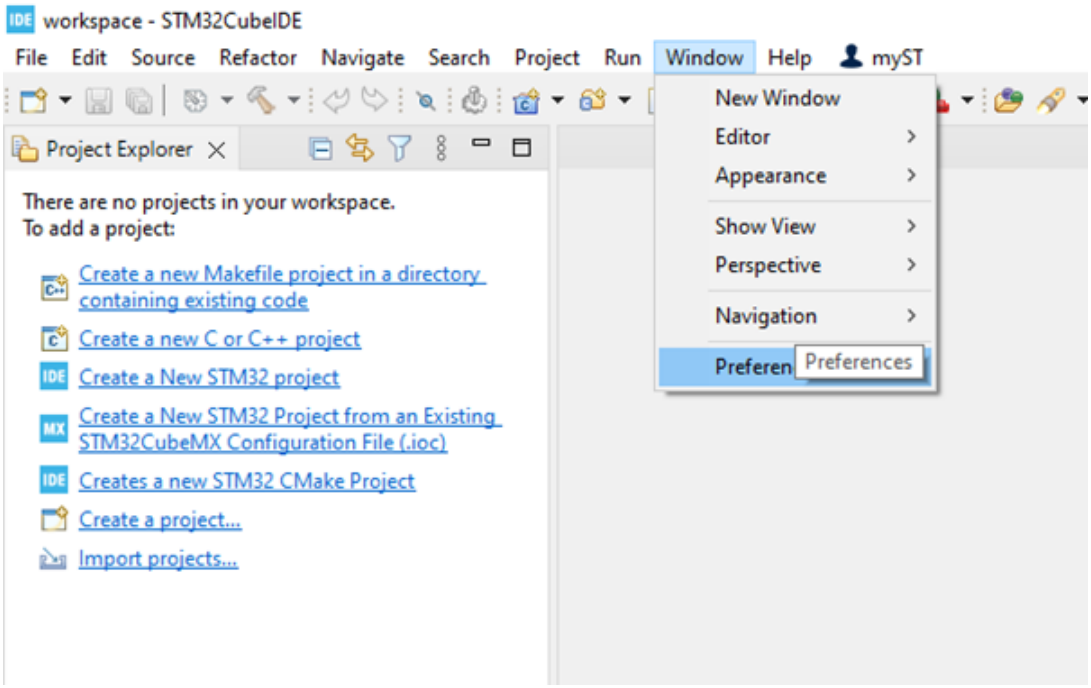


Figure 18: Lab PC setup step 1.

> ⓘ *In any of these project generator software set-ups, the user must be cautious. The software typically uses comment delimiters to allow user code to be added, but this system is prone to deleting user code if the project is regenerated (adding new I/O for instance). Our advice is to add the minimum code in the project generated files and to use the project generator sparingly (ideally once only when generating the project for the first time). We recommend that you add files, with calls originating from the main C-file, which contains your user created code.*

For a small project such as this, adding the following minimum files will simplify your development:

- Header file with all global definitions and function prototypes;
- Source file with all your global variables;

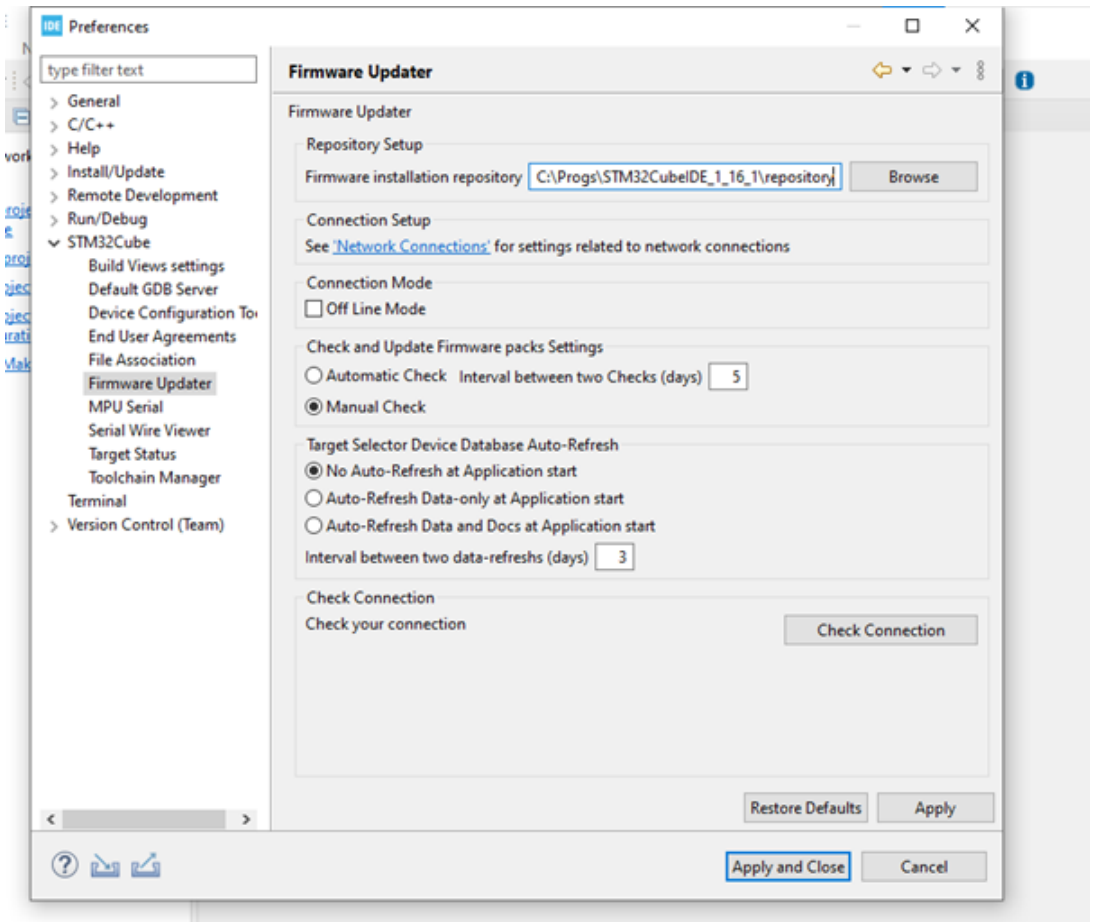


Figure 19: Lab PC setup step 2.

- Source file with initialisation function and user code, which leaves the main while loop short. Call a run function from main to execute your user code.
- Additional files to handle each mode, inputs (buttons), and LCD driver functions are recommended.

4.3 Using GIT for Design(E)-314

GIT is a form of source code version control. It not only keeps a backup of your code, but also allows you to easily see changes in the code between checked-in versions, and facilitates collaboration on source code projects (although we will not use this latter functionality). Having knowledge of GIT and its processes is beneficial since it is used almost everywhere in the industry where source code is part of the organization's Intellectual Property (IP). You will be ***required*** to use GIT and have your demo code (a separate version committed for each demo) in your GIT repository ***before*** you do your demonstration.

> *Please refer to the GIT document on STEMLearn for further details on how to set up and use GIT in this course.*

4.4 Design of your Software: Structure and timing

4.4.1 Functions and Loops

The next question the designer (you) will encounter is what software structure must be chosen. The suitable choices depend on the requirements at the system level and the peripheral response times.

Lets take an example approach:

The designer decides to put all the software in one big main loop with each function being executed

sequentially before repeating indefinitely. What could go wrong?

Let us explore the possible problems by asking some questions about the software's behaviour:

- How fast will the system respond to a UART command received? Will it consistently respond in this time?
- How fast will the system be able to sample digital inputs?
- How fast will the system be able to sample ADC inputs?
- How fast will the system respond to I2C inputs from the sensors?
- How long will the system take to do all the required calculations?
- How long will the system take to change the driver output parameters?
- How fast does the system NEED to do all the above, based on the specifications?
- Do all of these inputs/outputs require the same periodic attention from the system?
- How much resources will be consumed by the code? RAM, ROM, and stack?

There is also a second, more subtle, reason for making a good software structure choice at the start of the project: How will any code additions or changes in sub functions affect the rest of the code? What you don't want is to have to rewrite a big part of your main loop (and maybe some other functions too), to accommodate a new function's requirements.

> *Modularity and well-defined interfaces are key to keep the reworking of code to a minimum.*

In terms of the system design, we may attempt one of the following three approaches:

- The novice software writer will not consider a synchronous time-based structure for his code. Although it is feasible just to string all the functions together into one big while loop, the responsiveness of the system will be determined by the slowest function and recovery from time-outs and other errors is non-trivial to maintain.
- By using a timer-based schedule inside the while loop with interrupts can make the software much more robust. The choice of a tick-update period equal to 2× to 5× the largest response delay time will simplify the code (otherwise a state machine and elapsed timer will also be required).
- The use of a real-time operating system (RTOS), such as FreeRTOS, will simplify the structure further, but add some overheads. It also has a steep learning curve. In the simple case of each thread having the same priority, the behaviour will resemble a large while loop (equal priority from interrupt but with pre-defined execution order).

The large while loop (second option above), reacting to interrupt flags, is the preferred option for this project and learning phase.

4.4.2 HAL vs. LL Functions

A programmer may choose to write code at the register level or may want the abstraction that the HAL (Hardware Abstraction Layer) provide. The CubeMX initialisation generator will generate code for either the HAL or the LL (Low Level) libraries, but not register-level code directly. By using the CubeMX generator, the student can only choose LL or HAL code (per peripheral), and with the documentation bias towards HAL code, it is likely that most users will start off with HAL code.

The majority of tasks for this type of project can be written using the HAL functions only. If required, direct register access is still available using the STM32F303 header file.

Interrupts are accessible by using calls with IT-ending (e.g. `HAL_UART_Receive_IT()`) and a default interrupt handler for every interrupt is generated (e.g. `USART1_IRQHandler()` in the `stm32f3xx_it.c` file). You can process the flags to determine the cause of the interrupt in this file, but the recommended call-back function (e.g. `HAL_UART_RxCpltCallback()`) is a better option.

4.4.3 Regular Interval Timing

The core ARM processor provides a SysTick timer as standard, which is set in the HAL library at 1 msec intervals, so there is no need to provide any additional timers for this simple purpose. A call-back from the SysTick timer will provide a 1-millisecond heartbeat. This is located in the STM32f4xx_it.c file.

```
1         void SysTick_Handler(void)
2         {
3             /* USER CODE BEGIN SysTick_IRQn 0 */
4             /* USER CODE END SysTick_IRQn 0 */
5             HAL_IncTick();
6             /* USER CODE BEGIN SysTick_IRQn 1 */
7             /* USER CODE END SysTick_IRQn 1 */
8         }
9
```

However, in some cases, you need to generate shorter duration timer delays, functions that might need sub-microsecond accuracy. Hardware timers should be used in such cases, and you will have to add your own code to the timer interrupt vectors and create your own functions to implement these delays. For example, you would want to call such a delay function:

```
1         // Wait for 5 microseconds
2         myUsecDelay(5); // call self-created u-sec accurate delay function
3
```

4.4.4 Main While Loop Statistics

Suppose we want to find out how long our loop times are for the main while loop. One way is to set up a timer (example of timer 7 here) with a prescaler that divides the 84 MHz clock frequency down to 1 microsecond intervals. The following code fragment illustrates how to do this:

```
while (1){
// Prepare timer 7 to get execution time statistics
__HAL_TIM_SET_COUNTER(&htim7, 0);  // Reset the counter
HAL_TIM_Base_Start(&htim7); // Start Timer7 to get cycle time in usec

userProcess(); // Run the user process (your own cyclic program)

// Gather execution time statistics
HAL_TIM_Base_Stop(&htim7);        // Stop Timer7
elapsedTime = __HAL_TIM_GET_COUNTER(&htim7);

if (elapsedTime > maxElapsedTime)        // Update the maximum stats
maxElapsedTime = elapsedTime;
if (elapsedTime < minElapsedTime)        // Update the minimum stats
minElapsedTime = elapsedTime;
// Average stats:  Discrete IIR filter with 1/100 bandwidth
aveElapsedTime  = 0.99 * aveElapsedTime + 0.01 * elapsedTime;

__WFI();        // Wait for the next interrupt
}
```

This code will provide minimum, maximum, and average cycle times (to give you an indication of the processor loading). With the regular SysTick interrupt occurring every millisecond, the example of an average elapsed time of 20 microseconds indicate a 2% average loading.

> ⚠ *To use the timer in this mode, load the period value with a large value (such as* `0xFFFF`*) as it will count up, if you leave it at the default of zero then the timer will not count!*

4.4.5 Auto code generation

The STM IDE provides functionality to auto-generate code. We advise you to be cautious as if you auto-generate code halfway through a project, it can delete some of your code. Make sure all your code is between the correct comments if you plan to use this feature.

> ⚠ *We recommend only doing this once at the start of your project! Back up your code frequently to avoid loss of code!*

4.5 Debug and status indicator LEDS

The debug LEDs will be used as system state indicators as follows:

- D2 - indicates Proximity Alarm (D2 flashing)
- D3 - indicates Impact Detected (D3 = ON) or Unsafe driving (D3 flashing)
- D4 - indicates Low-light Alarm (D4 flashing)
- D5 - indicates Uncomfortable Temperature Alarm (D5 flashing)

The flashing timing for the debug LEDs should be 500 ms ON and 500 ms OFF.

4.6 UART

The UART shall operate at a baud rate of 57600 bps, with 8 data bits, 1 parity bit, and one stop bit, using TTL signal levels. The parity used will be even parity.

UART messages make use of ASCII printable characters, to make it easy to interface with the student board debug connector using simple terminal programs, such as CoolTerm, Termite, Realterm, and Teraterm in the absence of a test station.

The format of the UART receive HAL functions is slightly at odds with how UART communication normally occurs The HAL functions are geared towards known fixed length packets, while in reality, we use variable length packets. To use the HAL receive function, we can set it up for single-byte mode using interrupts (in effect to prime the receiver before the byte arrives), and as soon as we received a byte we can re-prime the receiver.

For UART transmission we can use standard block transmits.

4.6.1 UART Protocol

The SB UART will be used to send and receive messages to and from the PC (TS).

The SB will output messages to the PC (TS) with the formats as in Table 3. Each message starts with a '@' character and ends with a '&' character followed by a newline character (ASCII character code 10, or indicated in C string escape notation as '\n'). In the response, each field is separated by a comma ',' character. Except for the *StdNum* message that identifies the student board, all the transmit messages are in response to a message request received from the PC (TS).

Command format	Description
`@Stat&\n`	Request SB status information
`@Log&\n`	Toggle logging to SD Card ON/OFF
`@Dump&\n`	Dump SD card file contents over UART to PC/TS
`@SetWarn C=Y&\n`	Set Warning Condition 'C' to condition 'Y' (Y=1 to set; Y=0 to reset)
`@SFD xxx.x;zzz.z&\n`	Sending Fuel Data (Fuel fill-up of `xxx.x` liter, `zzz.z` Vehicle Kilometers)
`@RFE&\n`	Request Fuel Efficiency
`@CLF&\n`	CLear File to erase SD card data

Table 2: UART RX Message fields - Commands from PC (or TS) to SB.

Command received	Response by SB
None - at Start-up	`*12345678#\n`

Stat	`YYYY/MM/DD HH:MM:SS\n`
	`Distance:  xx.x cm\n`
	`Temperature: xx.x °C\n`
	`Light:  xxxx lux\n`
	`X accel:  ±x.xx g\n`
	`Y accel:  ±x.xx g\n`
	`Z accel:  ±x.xx g\n`
	`Unsafe driving:  y\n`
	`Impact detected:  y\n`
	`Low-Light warning: y\n`
	`Proximity warning: y\n`
	`High Temperature:  y\n`
	`GPS lat:  ±xxx.xxxxx\n`
	`GPS long: ±xxx.xxxxx\n`

Dump	Line-by-line information as stored in the .csv file
SetWarn	No UART response
SFD	No UART response
RFE	`YYYY/MM/DD HH:MM:SS\n` `Fuel Eff: xx.x km/L xx.x L/100km\n`
CLF	No UART response

Table 3: UART TX Message fields - Log from SB to PC (or TS).

4.6.2 UART Value Representation

Stat command: The values returned after receiving the *Stat* command are a list of parameters reporting the status of the system at that exact instance of time. Each of the lines returned are exactly 21 characters in length with the last character being a '\n' character.

The x's shown in Table 3 indicate the number format and required accuracy that should be used. For example 123.45 would be indicated as xxx.xx. The rest of the information should be formatted verbatim as shown in Table 3 including the units (g, lux, cm, etc).

Please note that accelerations, as well as longitude/lattitude are the only values that can be negative.

The y's shown in Table 3 indicate the boolean format (1 for True and 0 for False) that should be used.

Log command: The `Log`-command serves to activate or deactivate the logging, with the format defined in Table 6.

Dump command: The `Dump`-command should read the .csv file line-by line from the SD card and "dump" the information via the UART to the PC/TS. The data received via the UART will thus be in the same format as specified in the SD-card log format shown in Table 6.

SetWarn command: The `SetWarn`-command serves to set or reset warning conditions. An integer value of 'Y=1' is used for setting the condition and an integer value of 'Y=0' for resetting the condition. 'C' is a integer value from 1 to 5 which specifies the warning condition.

C=1 'Unsafe driving' warning condition
C=2 'Impact detected' warning condition
C=3 'Low-light' warning condition
C=4 'Proximity' warning condition
C=5 'Uncomfortable temperature' warning condition

SFD command: The SFD-command serves to set the amount liters refuelled (when refuelling at a service station) as well as the distance travelled (in kilometers) since the previous refuelling. The command format of `SFD xxx.x;zzz.z` indicates that `xxx.x` liter of fuel has been refuelled while a distance of `zzz.z` km has been travelled since the previous refuelling. It is assumed that the vehicle fuel tank is refuelled to full capacity.

RFE command: The RFE-command is used to calculate and return the fuel efficiency. It is assumed that the SFD-command has been previously received and executed. The system must return the fuel efficiency in both kilometer per liter (km/L) and liter per 100km (L/100km).

CLF command: The CLF-command should delete the log file on the SD card. This should only be allowed when the logging is disabled. When logging is enabled after a CLF-command, a new log file should be created on the SD card and data stored therein.

4.7 ADC Conversions and measurements

You will use the internal ADC to measure the light intensity. The ADC can be set to be fast (as little as 2-3 microseconds conversion time), so it has an adequate sample rate to sample the input signal. As an engineer, you should consider which sample rate will be "fast enough".

The ADC can operate with as low as 1.5 clock cycles sampling for regular channels (if the impedance is sufficiently matched). To convert a single channel using the HAL library, we need to set up the channel, start the conversion, wait for completion, and convert the result into the required scaled value that represents the original sampled signal.

You can also use the ADC in DMA mode to transfer the data directly to memory, bypassing the CPU. This might not be required (given highly efficient code) but is recommended when having to sample fast and many times in succession.

4.8 Real-Time Clock

Your system will need to keep accurate time. There are several design drivers for this requirement. For example, recording the date and time when a warning condition occurs. There are various options to implement an accurate timekeeping reference.

The first option is to use a timer that runs on the system clock. This will only be as accurate as the system clock. The accuracy of the system clock is dependant on the source of the system clock. By default the internal RC oscillator is used as the system clock reference. This internal oscillator is adequate for most basic timing operations that do not need a high accuracy over a longer period. The RC circuit is also sensitive to temperature fluctuations. For this application the internal RC clock source is good enough for the sampling of signals and short interval interrupts.

However, for something like a Real-Time Clock (RTC), where you expect a few seconds of drift over many weeks, requires a more accurate solution. If you inspect the clock configuration tab in the CubeIDE configuration window, you will see the STM32F411RE uses the external 32.768kHz crystal oscillator as the clock source for the RTC. The crystal oscillator is much more stable over a larger temperature range, and by doing some calibration against a good clock reference, the implemented timekeeping system can be fairly accurate. Of course, a better crystal oscillator will provide a more accurate result.

The built-in HAL library for the RTC is mostly self-explanatory and fairly easy to use.

> *For your project you should set the RTC value to be default: 2026/02/26 22:12:42 at power on.*

4.9 Ultrasonic Sensor Distance function

You will need to write software function to continuously (at a specified interval) obtain a raw measurement from the ultrasonic sensor. The raw sensor measurement must then be used to calculate the distance in **cm** to the object in the field of view of the ultrasonic sensor.

4.10 MPU-6050 acceleration Sensor driver

You will need to write a low-level driver to communicate with the MPU-6050 module and transfer command and data to it via the I2C interface. Please refer to the datasheets for the MPU-6050 module that is available on STEMLearn. You will need to continuously (at a specified interval) obtain raw acceleration measurements from the MPU-6050 module and convert these raw measurements into acceleration measured in **g** (9.81 $[m.s^{-2}]$).

4.11 Temperature Sensor driver

You will need to write software function to continuously (at a specified interval) obtain a raw measurement from the temperature sensor. The raw sensor measurement must then be used to calculate the ambient temperature in $^{\circ}C$.

4.12 Light Sensor driver

You will need to write software function to continuously (at a specified interval) obtain a raw measurement from the light sensor. The light sensor is an analogue sensor and you will thus need to use the ADC to sample the analogue output signal (refer to Section 4.7) from the op-amp. You will also need to calibrate the sensor to convert the raw measurement from your sensor to a light intensity value in **lux**.

4.13 Keypad driver and interface

The keypad will be the main user input interface for your system during normal operation. The keypad software should implement the low-level driver to read the key presses from the keypad. There is no intelligent electronics in the keypad, just connections to each row and column pin of the keypad. If the user presses a key, it will close a connection between a specific row and column, indicating which key was pressed.

To enable your processor to read the key presses, you must check each row/column separately in time. This can be achieved by sequentially driving each row/column high (one at a time) and then "scanning" the input row/column to see if any of the row/columns are being driven high (active high). If a row/column is detected as high, the combination of the row/column number with the driven row/column at that time will give you the information required to determine which key was pressed.

> ⚠ *All the keys on the keypad are effectively buttons that are pushed, and as such can cause button bounce on your inputs. You need to debounce the inputs from the keypad to avoid false double-presses being detected due to the bouncing signal.*

The system also requires functionality to adjust system settings and parameters via the keypad input. The display and system should respond to keypad inputs in a specified way. The specified display information and menu traversal steps are listed in table 4.

> ⓘ *Hint: Use table 4 to design a flowchart/state machine to manage your menu traversal and display pages as directed by the keypad inputs.*

There are 3 top-level menu pages: 'Measurements', 'Data Entry', 'Diagnostics'. The 'Measurements' menu page is the default page that is displayed when the system is started. A menu is entered by pressing the

RIGHT-S4 (→) button and a menu is exited by pressing the LEFT-S2 (←) button. A user cycles through the top-level menus by pressing the UP-S1 button (↑) or the DOWN-S5 button (↓). Each of the top-level menus consists of a number of pages. The 'Data Entry' menu allows a user to change system settings or input data. The 'Measurements' menu displays various system measurements. The 'Diagnostics' menu displays the status of the various subcomponents of the system.

When a key or button is pressed to change to a different display page or menu, the display and system state should stay in that changed state until the next key or button is pressed. The display and system state should not automatically change back to any assumed state without user input (i.e. do not implement a display or menu timeout).

Table 4: LCD display table for the user interface.

Menu Level	Action	Display LCD
1	display page *default page*	`= Measurements =` `Press -> display`
2	data entry page	`== Data Entry ==` `Press -> display`
3	diagnostics page	`== Diagnostics =` `Press -> display`
1-1	display page 1	`Dist:  xxx.x cm` `Temp:  xx.x °C`
1-2	display page 2	`Accel:  x.xx g` `Light:  xxxx lux`
1-3	display page 3	`Lat:  xx.xxxxxx°` `Long: xx.xxxxxx°`
1-4	display page 4	`Head:  xxx°` `Speed:xxx.x km/h`
1-5	display page 5	`Fuel Efficiency:` `xx.x L/km`
2-1	data entry page 1	`Enter fuel liters` `S3 for data entry`
2-2	data entry page 2	`Enter Odometer km` `S3 for data entry`
2-3	data entry page 3	`Log data (Y/N)` `'*' = Y / '#' = N`

3-1	diagnostics page 1
	SD card ready
	SD card not ready
		`-- Diagnostics --`
		`SD-card: OK`
		`SD-card: NOT OK`

3-2	diagnostics page 2
	MPU-6050 ready
	MPU-6050 not ready
		`-- Diagnostics --`
		`MPU-6050: OK`
		`MPU-6050: NOT OK`

3-3	diagnostics page 3
	GPS ready
	GPS not ready
		`-- Diagnostics --`
		`GPS: OK`
		`GPS: NOT OK`

3-4	diagnostics page 1
	SD card ready
	SD card not ready
		`-- Diagnostics --`
		`Logging Data: ON`
		`Logging Data: OFF`

The LCD should also be capable of delivering a 'Warning' screen, which should interrupt the default display page. When a 'Warning' appears, the user will be required to acknowledge the warning by pressing the S3 Button (middle button). Table 5 lists the 'Warnings'.

Each warning has to appear under certain conditions and should terminate or activate at different time intervals.

- Unsafe Driving: Activates when the measured acceleration (as recorded by the accelerometer, but excluding gravity) exceeds a value of 0.5 g in any direction. Once terminated with the S3 button, the warning can immediately appear again without any time delay if driving conditions exceed the limit.
- Low Light: Activates when light conditions are recorded at 300 Lux or lower. Warning to be terminated with S3 button. Once terminated with S3 button, the warning should not appear until 60 minutes have passed since Low Light Warning termination. This is to avoid the error from continuously repeating during night time conditions.
- Proximity Dist.: Activates when the ultrasonic sensor records a distance of 10 cm or lower. This warning can be terminated with either the S3 button or when the measured distance exceeds 30 cm, whichever comes first. The warning should appear continuously and there is no time limit delay between low distance warnings, since this is a high risk warning that requires intervention.
- High Temp.: Activates when measured temperature exceeds 30 °C. Warning to be terminated with S3 button or when the measured temperature falls below 30 °C. Once terminated, the warning should not appear until 30 minutes have passed since the High Temperature Warning termination. This is to avoid the error from continuously repeating while temperature conditions are being changed with the air-conditioning system or open windows.

Table 5: LCD WARNING display table for the user interface.

Warning Message	Action	Display LCD
1	display page	`== WARNING ==` `Unsafe Driving`
2	display page	`== WARNING ==` `Low Light`
3	display page	`== WARNING =` `Proximity Dist.`
4	display page	`== WARNING ==` `High Temp.`

4.14 LCD display libraries and interface

The objective of the LCD screen is to present useful information to the user.

> *Make sure to read the LCD datasheets provided for assistance in coding the initialisation and commands for the LCD.*

The LCD software should contain two levels of abstraction. The first and lower level is the driver. This is used to communicate with the LCD module at a low level and transfer command and data to it. The LCD must be operated in 4-bit (nibble) mode as the D0-D3 lines are grounded on the main PCB. This design reduces the amount of GPIO lines needed for the LCD interface to only 7, but changes the complexity of the software driver a little. You will have to write the 2 separate nibbles of the commands and characters to the LCD, highest nibble first. The second and higher level is the application level where your code must manage the information to be displayed on the LCD display.

The LCD uses the Hitachi HD44780 controller/driver parallel IC. Therefore, to design your software driver, you should refer to the HD44780 datasheet. The datasheet contains important information regarding initialising and configuration of the LCD interface, and available command options.

There are open-source drivers available online in the form of libraries, and you can use them for your project. Please use the drivers with caution as they were not developed to be compatible with your software

structure. You might have to adapt your code or the library to achieve compatibility.

> *Hint: Your driver-level software should also manage the timing control to update the display information to the LCD. This functionality is usually not explicitly included in online driver libraries.*

Your LCD application must populate the data structures that you use to pass information to the LCD in such a way that you display the correct data based on the input and system state. A good control structure to achieve this is a state machine. The information provided in Section 4.13, Table 4 can be used to design your state machine.

4.15 SD Card: libraries, interface, and logging

4.15.1 Low-level interface

The MicroSD Card Module has an SPI interface and 6 pins (GND, VCC, MISO, MOSI, SCK, CS). Note that the CS pin is not part of the standard SPI pins. You must manage the CS pin separately. With a 3.3V power supply, it is suitable for various applications such as data logging, audio, video, and graphics. The board also includes a 3.3V regulator circuit for level conversion. Note, the SD card reader has a FAT file system requirement. (TIP: In your STM32 project, under categories, you should configure the Middleware and Software packs setting to be FATFS).

> ***Hint**: You should configure your SPI interface for the STM32 microprocessor. Here is an example video https://www.youtube.com/watch?v=PBIm8BCnbyQ&ab_channel=PRTechTalk to initialise the SD card reader module. However, feel free to utilise other sources as well.*

4.15.2 Data logging

The purpose of the SD card module is to record the vehicle sensor data data. When your board loses power, the system must maintain some critical information to ensure continued and uninterrupted operation of the SB.

You must save the following data to the SD card:

1. The date and time when the measurements were taken
2. The light intensity, in lux
3. The ambient temperature, in degrees Celcius
4. The distance to the closest object, in cm
5. The acceleration measurements in X, Y and Z axis, in g
6. The state of the warning conditions, i.e. impact warning, uncomfortable temperature warning, proximity warning, unsafe driving warning, low-light warning.
7. (Optionally) The GPS longitute and latititude.

Each line of the data logged to the SD-card should be a comma-separated-values (.csv) file with the data fields as listed in Table 6, with 1 sec intervals between line entries:

4.16 I2C Communications

The I2C communications on the STM32 family is well supported by the HAL libraries, which remove most of the complexity from the user. The STM32F411RE HAL libraries provide a variety of functions for read-

Date and time	Light (lux)	Temperature (°C)	Distance (cm)	X acceleration (g)	Y acceleration (g)	Z acceleration (g)	Unsafe driving	Impact warning	Low-light warning	Proximity warning	High Temperature
YYYY/mm/dd HH:mm:ss	xxxx	xx.x	xx.x	x.xx	x.xx	x.xx	y	y	y	y	y

Table 6: SD Card log file format

```
2026/04/29 17:05:58,451,28.5,15.2,-0.11,0.35,0.57,1,0,0,0,0
2026/04/29 17:05:59,130,30.1,10.5,-0.15,0.35,0.37,0,0,1,0,1
2026/04/29 17:06:00,742,33.4,05.4,0.15,0.35,1.57,1,1,0,1,1
```

Table 7: Example of 3 file entries

ing from and writing to I2C devices, including calls that allow repeated start, interrupt and even DMA transfers.

Even with all the functions provided, not all the calls required are provided (no blocking-type of calls with repeated start) and there can be bugs. The use of interrupt calls provide very little benefit in this application.

The writing and debugging of the functions will be daunting to new users, and the following strategy is proposed:

1. Start off with a single transfer — preferable a memory read (which is inherently an address write-repeated start-read transaction). The ideal memory location to read is the product identification register(s).
2. Develop an I2C write call (with repeated start, using the HAL blocking write call as basis), write to one of the configuration parameter registers and use the memory read listed above to read the written info back (to make sure the the write was successful).
3. Then develop an I2C read call (current address and with repeated start) and test as above.
4. Start to string commands together, and always make sure that you try to read the info back for checking purposes.
5. Use a oscilloscope, a low value on the SCL and/or SDA lines will quickly indicate a misplaced STOP token (it is more challenging to try and figure it out from the software only).

It is highly recommended that you use the **HAL_I2C_Mem_Read** and **HAL_I2C_Mem_Write** functions from the HAL-library, but you are free to try it your own way too.

4.17 (Optional) GPS module driver

The details of the optional GPS module driver will be detailed in a future version of the Project Definition.

5 Testing

Your student board (SB) will be tested during demo sessions, as specified in the module framework. The testing will occur in an automated fashion. Your SB will be plugged in to a test station (TS). The TS will provide power to your SB and generate a number of signals that are connected to your SB. The TS will also monitor and check certain signals from your SB.

5.1 Test Interface connector (TIC)

In order to facilitate testing, a test-interface connector (TIC) is provided by P13. This connector is designed to be stackable and consists of an Amphenol Bergstik 16-pin surface mounted connector soldered to the bottom of the baseboard (solder side). This connector mates with a 16-pin Amphenol Dubox connector on the Test-station board (top side) that will be used during demonstrations. The connections provide a power source (9 V), UART and 10 other connections during testing of the system.

The signal connections to the TIC is detailed in Table 8, and the pin numbering of J13, J3, and P13 are as shown in Figure 20.

> *From your board, you will have to route the signals listed in Table 8 to the test connector, P13, by making use of the solder connectors J3 and J13. The solder pads on J3 and J13 are routed to pins on the TIC (P13).*

> *You will be required to bridge the jumpers at J30 and J31 to ensure your board receives the 9V from the TIC.*

P13 pin	J3/J13 solder connection	Signal	Direction
1	J3 - 1,2	V_bat - 9 V Supply from test station	SB <-> TS
2	N/C	V_bat - 9 V Supply from test station	SB <-> TS
3	J3 - 3,4	UART transmit (TX from student board, RX of test station)	SB -> TS
4	J13 - 3,4	5V supply from student board (fed back to test station for verification/measurement)	SB -> TS
5	J3 - 5,6	UART receive (RX from student board, TX of test station)	SB <- TS
6	J13 - 5,6	3.3 V supply from student board (fed back to test station for verification/measurement)	SB -> TS
7	J3 - 7,8	Button UP-S1 (connected to TS for debug/verification)	SB <-> TS
8	J13 - 7,8	Button LEFT-S2 (connected to TS for debug/verification)	SB <- TS
9	J3 - 9,10	Button MIDDLE-S3 (connected to TS for debug/verification)	SB -> TS
10	J13 - 9,10	Button RIGHT-S4 (connected to TS for debug/verification)	SB <- TS
11	J3 - 11,12	Button DOWN-S5 (connected to TS for debug/verification)	SB -> TS
12	J13 - 11,12	Light sensor measurement as input to TS (ADC input)	SB -> TS
13	J3 - 13,14	GND	SB -> TS
14	J13 - 13,14	Digital out of Temperature sensor (LMT01) to TS	SB -> TS
15	N/C	GND	SB <-> TS
16	N/C	GND	SB <-> TS

Table 8: Test-interface Connector Pin definitions

> *Connections shown in grey in Table 8 are signals that are already routed on the PCB - you do not have to do anything to connect them.*

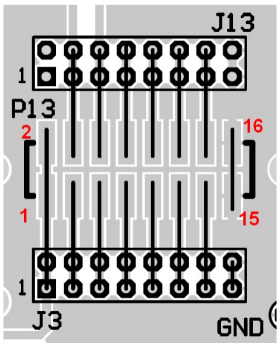


Figure 20: Pin numbers of J13, J3 and P13

5.2 Test method during demonstrations

The tests that the test station will execute (once a student board has been plugged in) are designed to verify the specifications of the student board (detailed in Section 2).

These specifications and the method by which the test station will perform the test will be detailed in a future version of the Project Definition.

5.3 UART communications interface

To facilitate testing (both by the student and for automated tests) and debugging of the device, a UART link shall be implemented. The UART link shall operate ***in both directions***: transmitted from the student board and received by the test station or PC test program. The student board shall make use of the default UART2 channel on the STM Nucleo - This UART channel is connected to the ST-Link chip on the Nucleo module, and will automatically enumerate as a virtual COM port when the Nucleo board is connected to the PC via the USB debug cable. Thus, for debugging it will not be necessary to make any hardware connections.

For test station purposes, it will be necessary to route the UART2 to the Test Interface Connector (TIC). In this case, you will have to make a connection from the correct pins of the Nucleo board (RX/TX) to the Test Interface Connector (TIC) (see Section 3.2 and 5.1). Both methods are shown in Figure 21.

> ⚠ *Remember that the RX/TX indicators are as seen from the ST-LINK perspective! If you are unsure, first check the direction with an oscilloscope, while sending messages from the PC, before connecting the wires to the TIC pins.*

5.4 Demonstrations

There will be a total of 4 demonstrations (demos) which will test all the system specifications. The details of the demonstrations will be provided in a future version of the Project Definition.

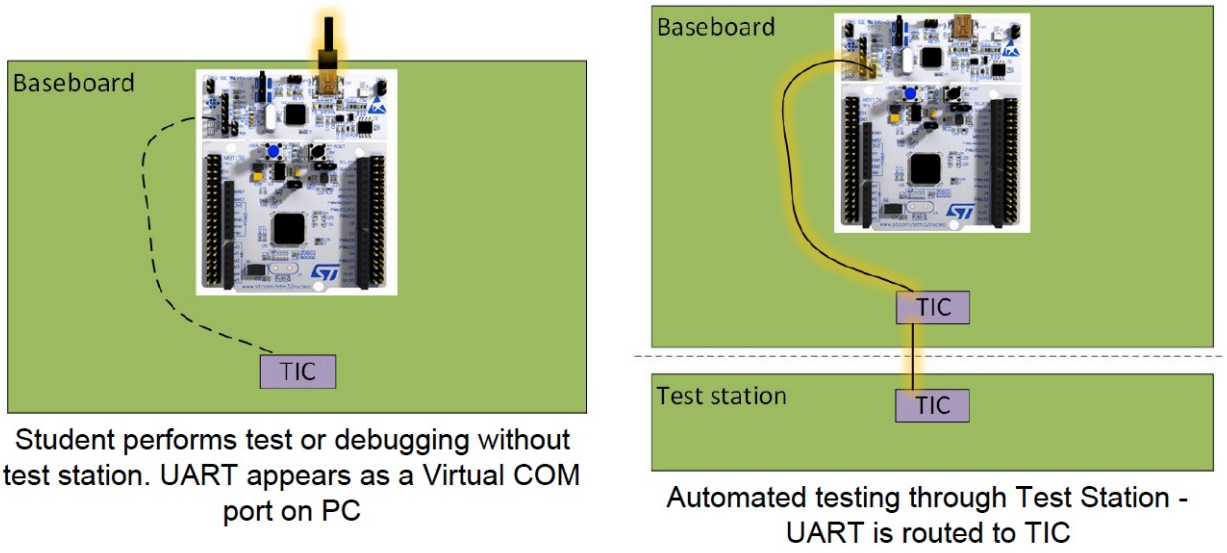


Figure 21: UART connections for debugging and testing.

6 Preliminary Report

The goal of the preliminary report is to give you exercise and feedback in writing parts of a technical report. You will mark your peers' reports, which should give you further insight into what is expected and some examples of interpretations of the tasks.

The details of the preliminary report will be provided in a future version of the Project Definition.

7 Final Report

The Final Report is due by the end of the semester and will be in electronic format and submitted to STEMLearn.

The details of the Final Report will be provided in a future version of the Project Definition.

References

[1] *RM0383 Reference Manual, STM32F411xC/E advanced Arm®-based 32-bit MCUs*, ST Microelectronics, Rev 4, May 2025.

[2] *NUCLEO-F411RE*, ST Microelectronics, https://www.st.com/en/evaluation-tools/nucleo-f411re.html, accessed 2026-02-03.

[3] *User manual STM32 Nucleo-64 boards (MB1136)*, ST Microelectronics, Rev 17, September 2025.